

Rider Management Plugin — Implementation Guide

Plugin: Nop.Plugin.Misc.RiderManagement
Branch: feature/plugin-setup-and-registration
Sprint: Sprint 1 — Rider Entity & Database Design
Date: May 20, 2026
Team: Team 7

Table of Contents

- 1. [User Story](#)
 - 2. [What Was Implemented](#)
 - 3. [File Changes Summary](#)
 - 4. [Database Design](#)
 - 5. [How to Run the Application](#)
 - 6. [Plugin Configuration & Activation](#)
 - 7. [How to Verify Everything Works](#)
 - 8. [Admin Panel Walkthrough](#)
 - 9. [Troubleshooting](#)
-

1. User Story

Rider Entity & Database Design

- Rider table created with fields: `Id` , `Name` , `Phone` , `Email`
 - `Status` (Active / Inactive)
 - `IsOnLine` (boolean)
 - Rider mapped with Customer entity (nopCommerce)
 - Repository & service layer implemented
 - Admin can view riders
 - Proper indexing for performance
 - Migration applied successfully
-

2. What Was Implemented

2.1 Domain Entity (Rider)

A new `Rider` entity was created that extends nopCommerce's `BaseEntity` . It stores delivery rider information and links to the nopCommerce `Customer` table via a foreign key.

Field	Type	Description
Id	int	Auto-incremented primary key (from BaseEntity)
Name	string	Full name of the rider (max 400 chars, required)
Phone	string	Phone number (max 50 chars, nullable)
Email	string	Email address (max 1000 chars, nullable)

StatusId	int	Mapped to RiderStatus enum (Active=0, Inactive=1)
IsOnline	bool	Whether the rider is currently available
CustomerId	int	Foreign key → nopCommerce Customer.Id

2.2 Database Migration

A FluentMigrator migration (`SchemaMigration`) automatically creates the `RiderManagement_Rider` table when the plugin is **installed** from the admin panel. The migration is tagged with `MigrationProcessType.Installation` so it runs once on install.

- **Table name:** `RiderManagement_Rider`
- **Index:** `IX_Rider_CustomerId` on the `CustomerId` column for performance

2.3 Repository Layer

nopCommerce's built-in generic `IRepository<T>` is used — no custom repository class needed. The framework auto-registers `IRepository<Rider>` via the Data layer, providing:

- `GetByIdAsync(id)` — fetch single rider
- `GetAllPagedAsync(query, pageIndex, pageSize)` — paged, filtered queries
- `InsertAsync(entity)` — insert
- `UpdateAsync(entity)` — update
- `DeleteAsync(entity)` — delete

2.4 Service Layer

Interface	Implementation	Location
<code>IRiderService</code>	<code>RiderService</code>	<code>Services/IRiderService.cs</code> / <code>Services/RiderService.cs</code>

Methods provided:

```

GetModuleGreetingAsync()           → string
GetAllRidersAsync(name, statusId, page, size) → IPagedList<Rider>
GetRiderByIdAsync(id)              → Rider
InsertRiderAsync(rider)            → void
UpdateRiderAsync(rider)            → void
DeleteRiderAsync(rider)            → void

```

`GetAllRidersAsync` supports filtering by **name** (contains search) and **status** (Active/Inactive), with pagination.

2.5 Admin Models

Model	Purpose
<code>RiderModel</code>	Create / Edit form fields
<code>RiderSearchModel</code>	Search filters for the list page
<code>RiderListModel</code>	Paged grid model (wraps <code>RiderModel</code>)

2.6 Admin Controller

RiderManagementController (area: Admin) provides:

Action	HTTP	Route	Description
Configure	GET	/Admin/RiderManagement/Configure	Plugin config page
List	GET	/Admin/RiderManagement/List	Rider list with search
RiderList	POST	/Admin/RiderManagement/RiderList	Ajax DataTables JSON endpoint
Create	GET	/Admin/RiderManagement/Create	New rider form
Create	POST	/Admin/RiderManagement/Create	Save new rider
Edit/{id}	GET	/Admin/RiderManagement/Edit/{id}	Edit rider form
Edit/{id}	POST	/Admin/RiderManagement/Edit/{id}	Save edited rider
Delete/{id}	POST	/Admin/RiderManagement/Delete/{id}	Delete rider

All actions are protected with `[AuthorizeAdmin]` and require the `MANAGE_PLUGINS` permission.

2.7 Admin Views

View	Description
Views/Admin/RiderList.cshtml	DataTables grid with name/status search filters and Add New button
Views/Admin/CreateOrEditRider.cshtml	Create/Edit form with Save, Save & Continue, and Delete buttons
Views/Configure.cshtml	Plugin configuration page (existing)

3. File Changes Summary

New Files Created

```
src/Plugins/Nop.Plugin.Misc.RiderManagement/
├─ Domains/
│   ├── Rider.cs                ← Rider entity
│   └─ RiderStatus.cs          ← Active/Inactive enum
├─ Data/
│   ├── Mapping/
│   │   ├── Builders/
│   │   │   └─ RiderBuilder.cs    ← Column definitions & FK index
│   │   └─ RiderNameCompatibility.cs ← Table name = RiderManagement_Rider
│   └─ Migrations/
│       └─ SchemaMigration.cs      ← Creates/drops the Rider table
├─ Models/
│   └─ Admin/
│       ├── RiderModel.cs          ← Create/Edit model
│       ├── RiderSearchModel.cs    ← Search filter model
│       └─ RiderListModel.cs       ← Paged grid model
└─ Views/
```

```
└─ Admin/
  └─ RiderList.cshtml          ← Admin list page
  └─ CreateOrEditRider.cshtml ← Admin create/edit page
```

Modified Files

```
Services/IRiderService.cs      ← Added full CRUD interface methods
Services/RiderService.cs       ← Full CRUD implementation using IRepository<Rider>
Controllers/RiderManagementController.cs ← Added List, Create, Edit, Delete actions
Infrastructure/RouteProvider.cs ← Added 4 new admin routes
RiderManagementDefaults.cs     ← Added route name constants
Views/_ViewImports.cshtml      ← Added Models.Admin, DataTables, TagHelpers.Admin usings
Nop.Plugin.Misc.RiderManagement.csproj ← Registered new view files as Content
```

4. Database Design

Table: RiderManagement_Rider

```
CREATE TABLE RiderManagement_Rider (
  Id          INT          NOT NULL IDENTITY PRIMARY KEY,
  Name        NVARCHAR(400) NOT NULL,
  Phone       NVARCHAR(50)  NULL,
  Email       NVARCHAR(1000) NULL,
  StatusId    INT          NOT NULL DEFAULT 0,
  IsOnline    BIT          NOT NULL DEFAULT 0,
  CustomerId  INT          NOT NULL,

  CONSTRAINT FK_Rider_CustomerId
    FOREIGN KEY (CustomerId) REFERENCES Customer(Id)
);

CREATE INDEX IX_Rider_CustomerId ON RiderManagement_Rider (CustomerId);
```

Note: The table is auto-created by FluentMigrator when the plugin is installed from the Admin panel. You do **not** need to run raw SQL.

Relationship with Customer

Each `Rider` row has a `CustomerId` FK pointing to the nopCommerce `Customer` table. This means every rider must have a corresponding nopCommerce customer account. This allows the rider to log in, be tracked by the existing customer system, and have access control.

5. How to Run the Application

Prerequisites

- .NET 10 SDK installed
- SQL Server running with the nopCommerce database already set up
- nopCommerce previously installed (first-run wizard completed)

Steps

1. Stop any running instance first (important before rebuilding)

```
# Find and stop any running Nop.Web process
Stop-Process -Name "Nop.Web" -Force -ErrorAction SilentlyContinue
```

2. Build the solution

```
cd C:\Users\KulwinderSingh\Desktop\nop_commerce-team-7\src
dotnet build NopCommerce.sln
```

Expected output:

```
Build succeeded.
    0 Error(s)
```

3. Start the application

```
cd C:\Users\KulwinderSingh\Desktop\nop_commerce-team-7\src\Presentation\Nop.Web
dotnet run
```

4. Open the browser

- **Storefront:** <https://localhost:44390> (or the port shown in terminal output)
- **Admin panel:** <https://localhost:44390/Admin>

6. Plugin Configuration & Activation

The plugin must be **installed** from the nopCommerce Admin panel before the database table is created and the admin menu appears.

Step-by-step Installation

1. Log in to the Admin panel: <https://localhost:<port>/Admin>
2. Navigate to **Configuration** → **Local plugins**
3. Find **"Rider Management"** in the list (System name: `Misc.RiderManagement`)
4. Click **"Install"**
5. Wait for the page to reload — the plugin is now active and the `RiderManagement_Rider` database table has been created automatically

Verify Plugin is Installed

- Go to **Configuration** → **Local plugins**
- The Rider Management plugin should show status **"Installed"**
- A **"Configure"** link should be visible next to it

Plugin Configuration Page

Navigate to: <https://localhost:<port>/Admin/RiderManagement/Configure>

This page shows:

- Current module status: "Rider Management module is active."
- Confirms the plugin is running correctly

No additional configuration is required for basic operation. The plugin works out of the box after installation.

7. How to Verify Everything Works

7.1 Verify Database Table Was Created

After installing the plugin, check your SQL Server database:

```
-- Check table exists
SELECT * FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_NAME = 'RiderManagement_Rider';

-- Check table structure
SELECT COLUMN_NAME, DATA_TYPE, IS_NULLABLE, CHARACTER_MAXIMUM_LENGTH
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_NAME = 'RiderManagement_Rider';

-- Check index exists
SELECT name, type_desc
FROM sys.indexes
WHERE object_id = OBJECT_ID('RiderManagement_Rider');
```

Expected: Table exists with 7 columns and index `IX_Rider_CustomerId` .

7.2 Verify Plugin Routes Are Registered

Visit these URLs while logged in as Admin:

URL	Expected Result
/Admin/RiderManagement/Configure	Plugin config page loads
/Admin/RiderManagement/List	Rider list grid loads
/Admin/RiderManagement/Create	Create rider form loads

7.3 Verify CRUD Operations

Create a Rider:

1. Go to `/Admin/RiderManagement/List`
2. Click **"Add New"** button
3. Fill in:
 - Name: `Test Rider`
 - Phone: `+1-555-0100`
 - Email: `rider@test.com`
 - Status: `Active`
 - IsOnline: checked
 - CustomerId: enter an existing customer ID from your database (e.g., `1`)

- 4. Click **"Save"**
- 5. You should be redirected to the list with a success notification

Read (List):

- 1. Go to `/Admin/RiderManagement/List`
- 2. The DataTables grid should show all riders
- 3. Test the search filters: type a name in the search box and click **Search**

Update:

- 1. In the list, click the **Edit** (pencil) button on any rider
- 2. Change a field
- 3. Click **"Save"**
- 4. Success notification should appear

Delete:

- 1. Open any rider's edit page
- 2. Click the **red "Delete"** button
- 3. Confirm the deletion dialog
- 4. Rider should be removed from the list

7.4 Check No Runtime Errors

After starting the app, watch the terminal output for:

- No `Exception` or `Error` messages related to `RiderManagement`
- Migration log should show: `"RiderManagement_Rider table created"` (on first install)

8. Admin Panel Walkthrough

Rider List Page (`/Admin/RiderManagement/List`)

+-----+
| Riders [+ Add New Rider] |
+-----+
| Search: [Name filter___] [Status: All ▼] [🔍 Search] |
+-----+
| Name | Phone | Email | Status | Online |
+-----+-----+-----+-----+-----+
| John Doe | 555-0100 | j@test.com | Active | ✓ |
| Jane Smith | 555-0200 | s@test.com | Inactive | |
+-----+

- **Name filter** — searches riders whose name contains the typed text
- **Status dropdown** — filter by All / Active / Inactive
- Status badges: green = Active, grey = Inactive
- Edit button opens the Create/Edit form

Create / Edit Rider Page

+-----+
| ← Riders [Save] [Save & Continue] [Delete] |

```

+-----+
| Name:      [ _____ ]
| Phone:     [ _____ ]
| Email:     [ _____ ]
| Status:    [ Active ▼ ]
| IsOnline:   [ ☒ ]
| CustomerId: [ ____ ] ← nopCommerce Customer ID
+-----+

```

Important: `CustomerId` must be a valid `nopCommerce Customer.Id`. You can find existing customer IDs at **Customers** → **Customers** in the admin panel.

9. Troubleshooting

Problem: Build fails with "file is locked by Nop.Web"

Cause: The application is still running when you try to build.

Fix:

```
Stop-Process -Name "Nop.Web" -Force -ErrorAction SilentlyContinue
dotnet build NopCommerce.sln
```

Problem: Rider table does not exist after installing the plugin

Cause: Migration did not run or plugin was not fully installed.

Fix:

1. Go to **Admin** → **Configuration** → **Local plugins**
 2. If the plugin shows "Installed", click **Uninstall**, then **Install** again
 3. Check the application logs at `src/Presentation/Nop.Web/App_Data/Logs/` for migration errors
-

Problem: Admin menu for Rider Management does not appear

Cause: Plugin not installed, or the `plugins.json` cache is stale.

Fix:

1. Ensure the plugin is installed (see Section 6)
 2. Check `src/Presentation/Nop.Web/App_Data/plugins.json` — `"Misc.RiderManagement"` should appear in the `"InstalledPlugins"` array
 3. Restart the application
-

Problem: "CustomerId" foreign key constraint violation when saving a rider

Cause: The `CustomerId` you entered does not exist in the `Customer` table.

Fix:

1. Go to **Admin** → **Customers** → **Customers**
2. Note the `Id` of an existing customer

3. Use that `Id` in the Rider form's `CustomerId` field

Problem: 404 on `/Admin/RiderManagement/List`

Cause: Plugin not installed, or routes not registered.

Fix:

1. Install the plugin from **Configuration** → **Local plugins**
 2. Restart the application: stop `dotnet run` and start again
-

Document generated: May 20, 2026 — Team 7